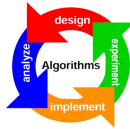


Estrategias algorítmicas

José A. Rodríguez Melquiades

26 de abril de 2026

Branch-and-Bound



1 Solución de modelos de PLE

2 Branch-and-Bound

- Algoritmo B & B

Solución de modelos de PLE

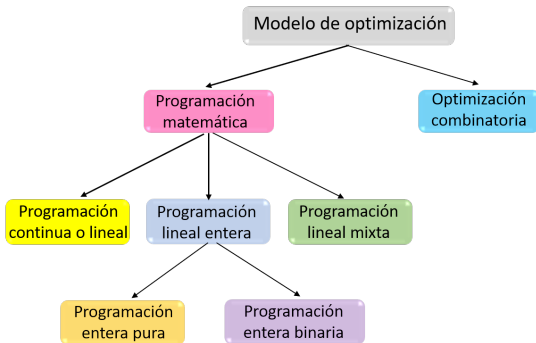
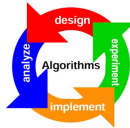


Figura 1: Modelos de optimización lineal

Branch-and-Bound



1 Solución de modelos de PLE

2 **Branch-and-Bound**

- Algoritmo B & B

Branch-and-Bound - BB

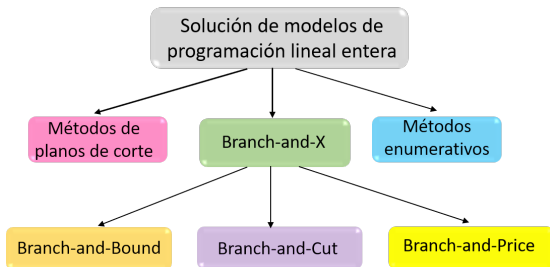


Figura 2: Métodos exactos para solucionar modelos de programación entera

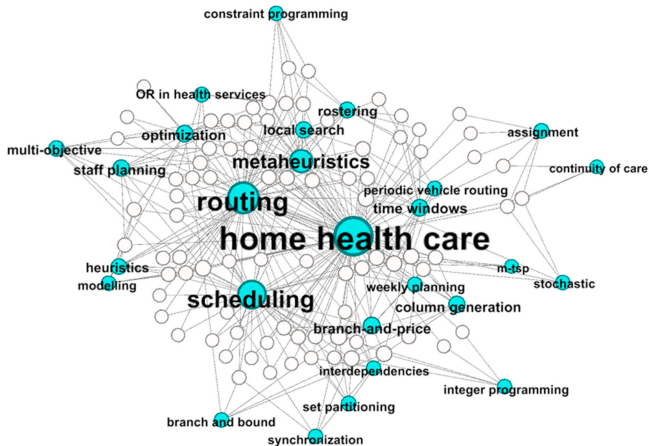


Figura 3: Métodos exactos y aproximados para solucionar el Home Health Care

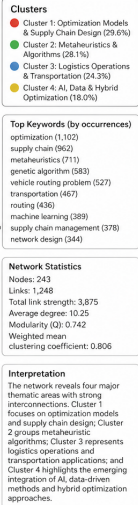
Logistics and Supply Chain Optimization
Model and Metaheuristics

Figura 4: Supply Chain: Optimization and metaheuristics

- Observación:

● Observación:

- ▶ **Teóricamente:** Cualquier PLE pura con variables enteras limitadas puede ser solucionado *enumerando todas las posibles combinaciones de valores enteros* y hallar una combinación (solución) que satisfaga las restricciones y produzca el objetivo.

● Observación:

- ▶ **Teóricamente:** Cualquier PLE pura con variables enteras limitadas puede ser solucionado *enumerando todas las posibles combinaciones de valores enteros* y hallar una combinación (solución) que satisfaga las restricciones y produzca el objetivo.
- ▶ **Desafortunadamente:** Si la cantidad total de combinaciones es muy grande, entonces no usar este proceso.

● Observación:

- ▶ **Teóricamente:** Cualquier PLE pura con variables enteras limitadas puede ser solucionado *enumerando todas las posibles combinaciones de valores enteros* y hallar una combinación (solución) que satisfaga las restricciones y produzca el objetivo.
- ▶ **Desafortunadamente:** Si la cantidad total de combinaciones es muy grande, entonces no usar este proceso.
 - ★ Si un problema tiene n variables enteras con m valores, entonces se tendrá un total de m^n *posibles combinaciones (soluciones viables y no viables)*. Por lo tanto, una enumeración completa es teóricamente simple pero intratable.

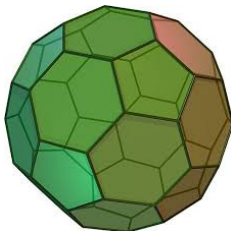


Figura 5: Espacio de soluciones

- B & B:

- B & B:

- ▶ Estrategia algorítmica exacta que soluciona problemas de optimización combinatoria y discreta.

- B & B:

- ▶ Estrategia algorítmica exacta que soluciona problemas de optimización combinatoria y discreta.
- ▶ Considerado como "dividir para conquistar".

- B & B:

- ▶ Estrategia algorítmica exacta que soluciona problemas de optimización combinatoria y discreta.
- ▶ Considerado como "dividir para conquistar".
 - ★ *Branching*: Dividir el problema.

- **B & B:**

- ▶ Estrategia algorítmica exacta que soluciona problemas de optimización combinatoria y discreta.
- ▶ Considerado como "dividir para conquistar".
 - ★ *Branching*: Dividir el problema.
 - ★ *Bounding*: Conquistar.

Tabla 1: Problemas solucionados con Branch-and-Bound

Problema	Descripción
Problema de la Mochila (0/1 Knapsack)	Dado un conjunto de objetos con pesos y valores, seleccionar un subconjunto que maximice el valor total sin exceder la capacidad de la mochila.
Problema del Viajante de Comercio (TSP)	Hallar un recorrido de mínimo costo visitando cada ciudad exactamente una vez y regresa al punto de origen.
Asignación (Assignment Problem)	Asignar n tareas a n agentes minimizando el costo total de asignación.
Programación Entera (General IP)	Maximizar o minimizar una función lineal sujeta a restricciones lineales con variables enteras.
Cobertura de Conjuntos (Set Covering)	Elegir el mínimo número de subconjuntos cuya unión cubra todos los elementos de un conjunto universal.
Problema de Partición	Dividir un conjunto en dos subconjuntos con sumas lo más iguales posible.
Árbol generador mínimo	Encontrar un árbol que conecte todos los nodos de un grafo con el mínimo costo total.
Ruteo de Vehículos (VRP)	Determinar rutas para una flota de vehículos que atiendan a todos los clientes con costo mínimo.
Maximización de Beneficio con Restricciones	Seleccionar proyectos o inversiones para maximizar el beneficio total sujeto a limitaciones de recursos.
Diseño de Horarios (Scheduling)	Asignar trabajos a máquinas o tiempos para optimizar un criterio (minimizar el tiempo total de finalización).

- Ejemplo:

$$\text{Max } Z = 3x_1 + 2x_2 \quad (1)$$

s.a.

$$2x_1 + 5x_2 \leq 9 \quad (2)$$

$$4x_1 + 2x_2 \leq 9 \quad (3)$$

$$x_1, x_2 \geq 0 \text{ y } \textit{entero} \quad (4)$$

● Ejemplo (Cont.)

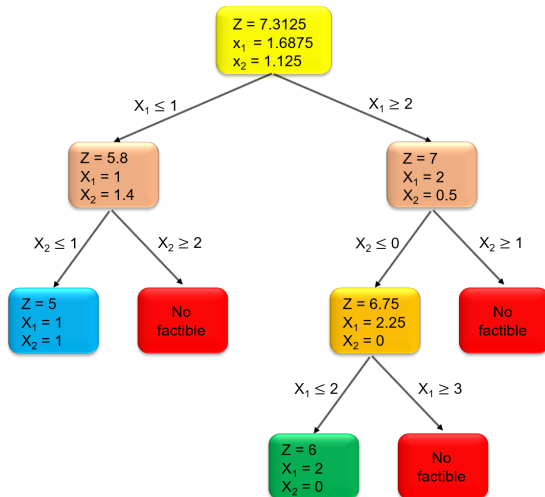


Figura 7: Problema original y subproblemas

- **Representación:** B & B es un árbol de enumeración especializado.

- **Representación:** B & B es un árbol de enumeración especializado.
 - ▶ **Nodo raíz:** Representa la solución del PL relajado para el problema de programación entero original (S_{PL}).

- **Representación:** B & B es un árbol de enumeración especializado.
 - ▶ **Nodo raíz:** Representa la solución del PL relajado para el problema de programación entero original (S_{PL}).
 - ★ **SI** la solución óptima del PL es entero, entonces problema resuelto.

- **Representación:** B & B es un árbol de enumeración especializado.
 - ▶ **Nodo raíz:** Representa la solución del PL relajado para el problema de programación entero original (S_{PL}).
 - ★ **SI** la solución óptima del PL es entero, entonces problema resuelto.
 - ★ **CASO CONTRARIO**, el valor objetivo del PL será el *upper bound* inicial sobre el valor objetivo óptimo del problema de PE. Por tanto el nodo raíz se particiona en dos nodos (subproblemas) mediante dos ramas.

- **Representación:** B & B es un árbol de enumeración especializado.
 - ▶ **Nodo raíz:** Representa la solución del PL relajado para el problema de programación entero original (S_{PL}).
 - ★ **SI** la solución óptima del PL es entero, entonces problema resuelto.
 - ★ **CASO CONTRARIO**, el valor objetivo del PL será el *upper bound* inicial sobre el valor objetivo óptimo del problema de PE. Por tanto el nodo raíz se particiona en dos nodos (subproblemas) mediante dos ramas.
 - ▶ **Ramas:** Son cortes válidos en términos de restricciones de desigualdad simple. Sus propiedades son:
 - (a) Cortan el punto óptimo actual de PL no entero y otra región fraccionaria.
 - (b) Los dos nodos sucesores son mutuamente excluyentes y su unión contiene la misma región factible entera que la de su predecesor, es decir, no se eliminan valores enteros.

- Ejemplo:

$$\text{Maximizar } Z = 5y_1 - 2y_2$$

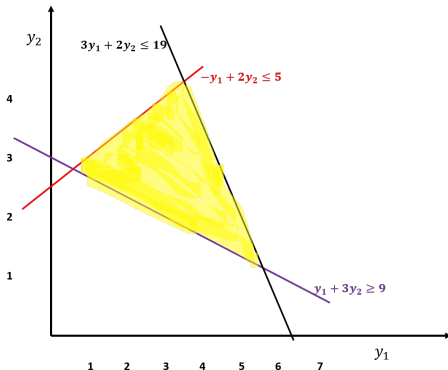
s.t.

$$-y_1 + 2y_2 \leq 5$$

$$3y_1 + 2y_2 \leq 19$$

$$y_1 + 3y_2 \geq 9$$

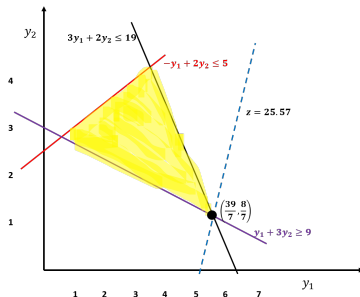
$$y_1, y_2 \geq 0, \text{ entero}$$



- Ejemplo: (cont.)

- Ejemplo: (cont.)

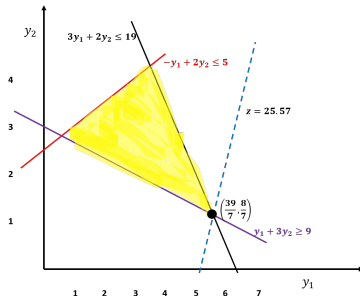
- ▶ Relajación S_{LP} :



- ▶ Optimo no entero: $y_1 = 39/7 = 5,57$ $y_2 = 8/7 = 1,14$ $Z = 25,57$

- Ejemplo: (cont.)

- ▶ Relajación S_{LP} :

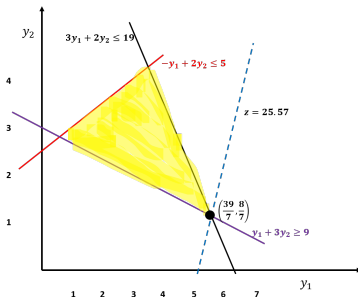


- ▶ Óptimo no entero: $y_1 = 39/7 = 5,57$ $y_2 = 8/7 = 1,14$ $Z = 25,57$

- ▶ Límite superior(Upper bound): $Z = 25,57$

- Ejemplo: (cont.)

- ▶ Relajación S_{LP} :



- ▶ Optimo no entero: $y_1 = 39/7 = 5,57$ $y_2 = 8/7 = 1,14$ $Z = 25,57$
- ▶ Límite superior(Upper bound): $Z = 25,57$
- ▶ Límite inferior(Lower bound): ∞ .

- Ejemplo: (cont.)

- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S'

- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S'

- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.

SubProblema 1:

$$\text{Maximizar } Z = 5y_1 - 2y_2$$

s.t.

$$-y_1 + 2y_2 \leq 5$$

$$3y_1 + 2y_2 \leq 19$$

$$y_1 + 3y_2 \geq 9$$

$$y_1 \geq 6$$

$$y_1, y_2 \geq 0, \text{ entero}$$

SubProblema 2:

$$\text{Maximizar } Z = 5y_1 - 2y_2$$

s.t.

$$-y_1 + 2y_2 \leq 5$$

$$3y_1 + 2y_2 \leq 19$$

$$y_1 + 3y_2 \geq 9$$

$$y_1 \leq 5$$

$$y_1, y_2 \geq 0, \text{ entero}$$

- Ejemplo: (cont.)

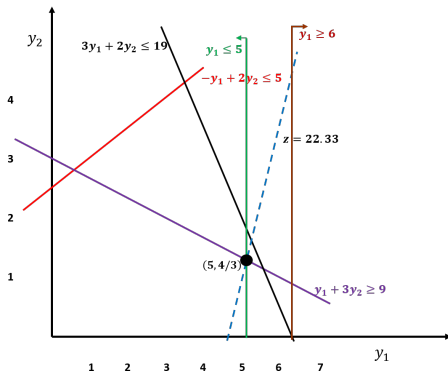
- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

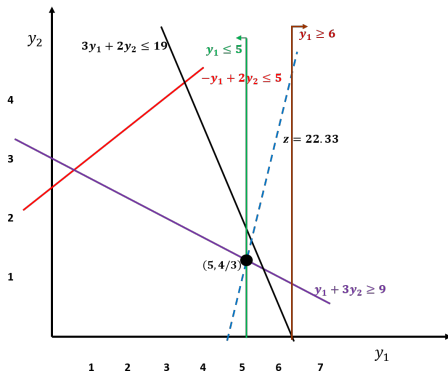
- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.



- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

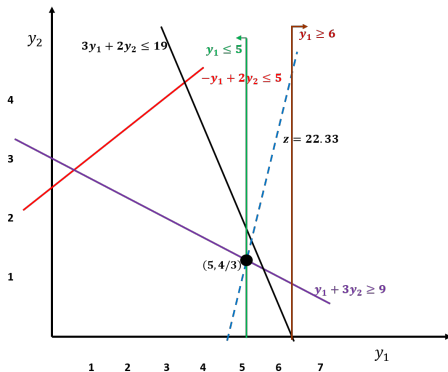
- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.



- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.

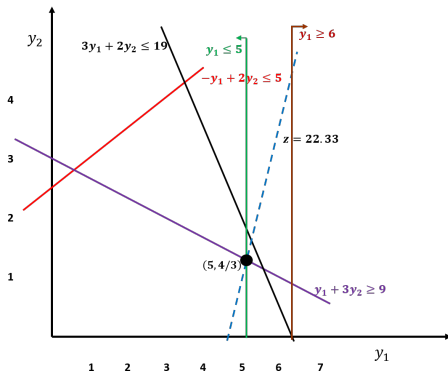


- ★ Rama con la restricción agregada $y_1 \geq 6$: Inviabile.

- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.

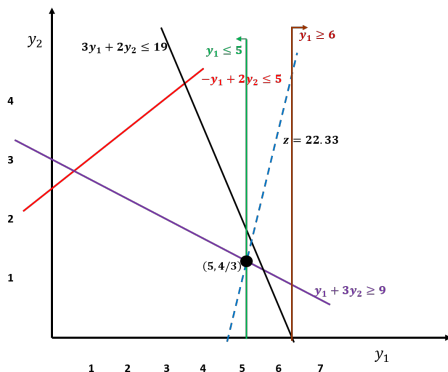


- ★ Rama con la restricción agregada $y_1 \geq 6$: Inviabile.
- ★ Rama con restricción agregada $y_1 \leq 5$: Óptimo en $(y_1, y_2) = (5, 4/3)$; $Z = 22,33$.

- Ejemplo: (cont.)

- ▶ Seleccionar arbitrariamente y_1 como la variable que será ramificada: S' (cont.)

- ★ Se generan dos subproblemas con las restricciones $y_1 \geq 6$ y $y_1 \leq 5$.



- ★ Rama con la restricción agregada $y_1 \geq 6$: Inviabile.

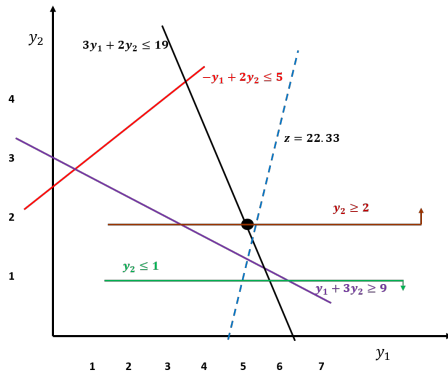
- ★ Rama con restricción agregada $y_1 \leq 5$: Óptimo en $(y_1, y_2) = (5, 4/3)$; $Z = 22,33$.

- ▶ Límite superior(Upper bound): $Z = 22,33$

- Ejemplo: (cont.)

● Ejemplo: (cont.)

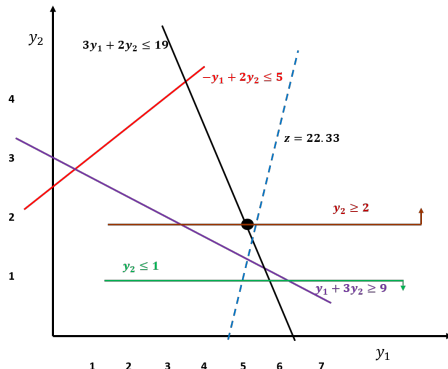
- ▶ Seleccionar y_2 para crer nueva rama: S''



● Ejemplo: (cont.)

► Seleccionar y_2 para crer nueva rama: S''

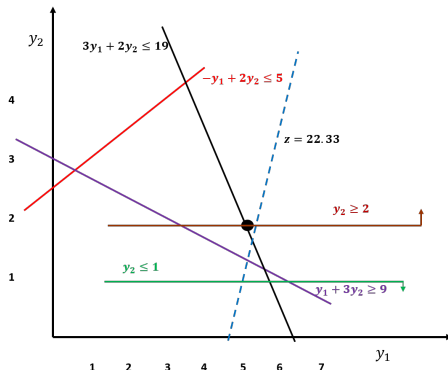
★ Se generan dos subproblemas agregando las restricciones $y_2 \geq 2$; $y_2 \leq 1$



● Ejemplo: (cont.)

► Seleccionar y_2 para crer nueva rama: S''

★ Se generan dos subproblemas agregando las restricciones $y_2 \geq 2$; $y_2 \leq 1$

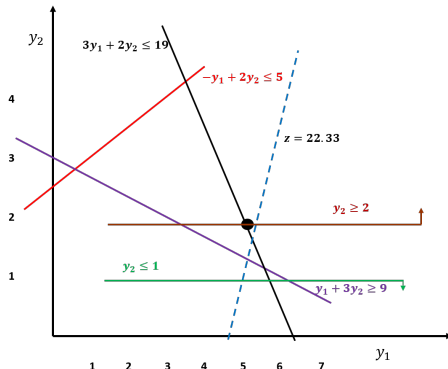


★ Rama con restricción agregada $y_2 \leq 1$: Inviabile.

● Ejemplo: (cont.)

► Seleccionar y_2 para crer nueva rama: S''

- ★ Se generan dos subproblemas agregando las restricciones $y_2 \geq 2$; $y_2 \leq 1$



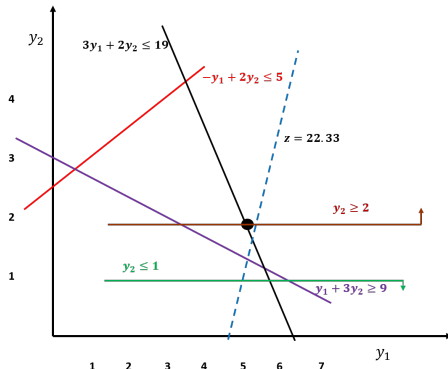
- ★ Rama con restricción agregada $y_2 \leq 1$: Inviabile.

- ★ Rama con restricción agregada $y_2 \geq 2$: Óptimo en $(y_1, y_2) = (5, 2)$; $Z = 21$.

● Ejemplo: (cont.)

► Seleccionar y_2 para crer nueva rama: S''

★ Se generan dos subproblemas agregando las restricciones $y_2 \geq 2$; $y_2 \leq 1$



★ Rama con restricción agregada $y_2 \leq 1$: Inviabile.

★ Rama con restricción agregada $y_2 \geq 2$: Óptimo en $(y_1, y_2) = (5, 2)$; $Z = 21$.

► Límite inferior(Lower bound): $Z = 21$. Es óptimo.

- Ejemplo: (cont.)

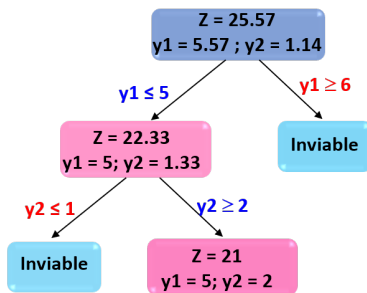


Figura 8: Árbol solución del PLE formulado

- Nota:

- **Nota:**

- ▶ Cuando una solución para un PL tiene muchas variables con valor fraccionario, es necesario decidir que variable será elegida para formar una rama. Las siguientes reglas se usan para elegir una variable para formar nuevas ramas:

- **Nota:**

- ▶ Cuando una solución para un PL tiene muchas variables con valor fraccionario, es necesario decidir que variable será elegida para formar una rama. Las siguientes reglas se usan para elegir una variable para formar nuevas ramas:
 - ★ Variable con valor fraccionario más cercano a 0.5.

● **Nota:**

- ▶ Cuando una solución para un PL tiene muchas variables con valor fraccionario, es necesario decidir que variable será elegida para formar una rama. Las siguientes reglas se usan para elegir una variable para formar nuevas ramas:
 - ★ Variable con valor fraccionario más cercano a 0.5.
 - ★ Variable con mayor impacto en la función objetivo.

● **Nota:**

- ▶ Cuando una solución para un PL tiene muchas variables con valor fraccionario, es necesario decidir que variable será elegida para formar una rama. Las siguientes reglas se usan para elegir una variable para formar nuevas ramas:
 - ★ Variable con valor fraccionario más cercano a 0.5.
 - ★ Variable con mayor impacto en la función objetivo.
 - ★ Variable con el menor índice.

● **Nota:**

- ▶ Cuando una solución para un PL tiene muchas variables con valor fraccionario, es necesario decidir que variable será elegida para formar una rama. Las siguientes reglas se usan para elegir una variable para formar nuevas ramas:
 - ★ Variable con valor fraccionario más cercano a 0.5.
 - ★ Variable con mayor impacto en la función objetivo.
 - ★ Variable con el menor índice.
- ▶ El objetivo es minimizar el número total de nodos evaluados en el árbol B&B. El rendimiento de estas reglas de bifurcación depende de la estructura del problema.

Algoritmo B & B

- Notación

Algoritmo B & B

- Notación

- ▶ S: Problema de programación entera (PE).

Algoritmo B & B

- Notación

- ▶ S : Problema de programación entera (PE).
- ▶ S_{PL} : Relajación PL de S .

Algoritmo B & B

- Notación

- ▶ S : Problema de programación entera (PE).
- ▶ S_{PL} : Relajación PL de S .
- ▶ y_{PL} : Solución para la relajación PL del PE dado.

Algoritmo B & B

● Notación

- ▶ S : Problema de programación entera (PE).
- ▶ S_{PL} : Relajación PL de S .
- ▶ y_{PL} : Solución para la relajación PL del PE dado.
- ▶ $\bar{Z}$: El menor (mejor) límite superior sobre Z^* del problema PE dado.

Algoritmo B & B

● Notación

- ▶ S : Problema de programación entera (PE).
- ▶ S_{PL} : Relajación PL de S .
- ▶ y_{PL} : Solución para la relajación PL del PE dado.
- ▶ $\bar{Z}$: El menor (mejor) límite superior sobre Z^* del problema PE dado.
- ▶ $\underline{Z}$: El mayor (mejor) límite inferior sobre Z^* del problema PE dado.

Algoritmo B & B

- Notación

- ▶ S : Problema de programación entera (PE).
- ▶ S_{PL} : Relajación PL de S .
- ▶ y_{PL} : Solución para la relajación PL del PE dado.
- ▶ $\bar{Z}$: El menor (mejor) límite superior sobre Z^* del problema PE dado.
- ▶ $\underline{Z}$: El mayor (mejor) límite inferior sobre Z^* del problema PE dado.

- Los límites son periódicamente actualizados según las ramas que se van formando, es decir, caminos sobre el árbol.

- Notación (cont.)

- Notación (cont.)

- ▶ S^k : Subproblema k del problema S .

- Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .

- Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z^k}$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z}^k$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).
- ▶ $\underline{Z}^k$: Mejor (mas alto) límite inferior de S^k (Nodo anteriormente mostrado k).

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z}^k$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).
- ▶ Z^k : Mejor (mas alto) límite inferior de S^k (Nodo anteriormente mostrado k).
- ▶ y_{PL}^k : Solución óptima del subproblema PL S_{PL}^k .

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z}^k$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).
- ▶ Z^k : Mejor (mas alto) límite inferior de S^k (Nodo anteriormente mostrado k).
- ▶ y_{PL}^k : Solución óptima del subproblema PL S_{PL}^k .
- ▶ $\bar{y}_j$: Valor no entero de una variable entera y_j (valor numérico actual de y_j).

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z}^k$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).
- ▶ Z^k : Mejor (mas alto) límite inferior de S^k (Nodo anteriormente mostrado k).
- ▶ y_{PL}^k : Solución óptima del subproblema PL S_{PL}^k .
- ▶ $\bar{y}_j$: Valor no entero de una variable entera y_j (valor numérico actual de y_j).
- ▶ $\lfloor a \rfloor$: Mayor entero $\leq a$ (o redondeo al inmediato inferior a).

● Notación (cont.)

- ▶ S^k : Subproblema k del problema S .
- ▶ S_{PL}^k : Relajación PL del subproblema k .
- ▶ Z^k : Valor objetivo óptimo de S^k .
- ▶ $\overline{Z}^k$: Mejor (mas menor) límite superior de S^k (Nodo anteriormente mostrado k).
- ▶ Z^k : Mejor (mas alto) límite inferior de S^k (Nodo anteriormente mostrado k).
- ▶ y_{PL}^k : Solución óptima del subproblema PL S_{PL}^k .
- ▶ $\bar{y}_j$: Valor no entero de una variable entera y_j (valor numérico actual de y_j).
- ▶ $\lfloor a \rfloor$: Mayor entero $\leq a$ (o redondeo al inmediato inferior a).
- ▶ $\lceil a \rceil$: Menor entero $\geq a$ (o redondeo al inmediato superior a).

● Algoritmo B& B:

- (0) **Inicialización:** Solucionar el PL relajado (S_{PL}) del problema PE dado (S).
SI es inviable, entonces el problema PE termina.
SI la solución óptima del PL satisface el requerimiento entero, el problema PE esta solucionado y por tanto termina.
CASO CONTRARIO, inicializar el mejor límite superior ($\bar{Z}$) con el valor objetivo óptimo del problema S_{PL} y el mejor límite inferior por $\underline{Z} = -\infty$. Colocar S_{PL}^k sobre la lista activa de nodos(subproblemas).
- (1) **Elección de un nodo:** **IF** lista activa esta vacia, terminar. La solución y^* es optimal.
CASO CONTRARIO, escoger un nodo subproblema S^k con S_{PL}^k con una de las reglas.
- (2) **Actualizar el límite superior:** Solucionar y un conjunto $\bar{Z}^k$ igual a valor objetivo del LP. Mantener guardar la solución óptima de y_{PL}^k .
- (3) **Poda por inviabilidad:** **IF** S_{PL}^k no tiene solución viable, apodar el nodo actual e ir al paso 1. **CASO CONTRARIO**, ir al paso 4.

● Algoritmo B& B: (cont.)

(4) **Podar por límite:** IF $Z^k \leq \underline{Z}$, apodar el nodo actual e ir al paso 1. **CASO CONTRARIO**, ir al paso 5.

(5) **Actualizar límite inferior y poda por optimalidad:**

(a) IF el óptimo del PL y_{PL}^k es entero, la solución viable para S se ha hallado.

Establecer $\underline{Z}^k = y_{PL}^k$ y comparar $\underline{Z}^k$ con $\underline{Z}$.

IF $\underline{Z}^k > \underline{Z}$, establecer $\underline{Z} = \underline{Z}^k$. **CASO CONTRARIO** $\underline{Z}$ no cambia. El nodo actual es apodado, pues no se puede obtener una mejor solución desde este nodo. Ir al paso 1.

(b) IF el óptimo del PL y_{PL}^k no es entero, ir al paso 6.

(6) **Ramificando:** Desde el nodo actual S^k escoger la variable y_j que tiene un valor fraccional, para generar dos subproblemas S_1^k y S_2^k definido por:

$$S_1^k = S^k \cap \{y : y_j \leq \lfloor \bar{y}_j \rfloor\}$$

$$S_2^k = S^k \cap \{y : y_j \geq \lceil \bar{y}_j \rceil\}$$

Colocar ambos nodos en una lista activa e ir al paso 1.